

Local RAG System - Sample Knowledge Base

This document is designed as a small knowledge base for testing a local Retrieval-Augmented Generation (RAG) application built with LangChain, ChromaDB, SentenceTransformers, and Ollama.

1. What is Retrieval-Augmented Generation?

Retrieval-Augmented Generation, commonly called RAG, combines information retrieval with a language model. Instead of asking the language model to rely only on knowledge learned during training, the application first searches a document collection for relevant information. The retrieved text is then provided to the language model as context for answering the user's question.

A basic RAG workflow contains two major stages. During ingestion, documents are loaded, divided into chunks, converted into embeddings, and stored in a vector database. During question answering, the user's query is embedded, similar chunks are retrieved, and those chunks are inserted into the prompt sent to the language model.

2. Document Loading and Chunking

Documents should be loaded together with useful metadata. For a PDF, metadata can include the source filename and page number. Preserving this information allows the final application to show where retrieved information came from.

Large documents are divided into smaller chunks before embedding. Chunking helps retrieval because a query can match a focused passage instead of an entire document. A small overlap between neighboring chunks can preserve context that crosses a chunk boundary. Extremely small chunks may lose meaning, while extremely large chunks can introduce irrelevant information.

3. Embeddings and ChromaDB

An embedding model converts text into numerical vectors. Texts with similar meanings should have vectors that are close in the embedding space. The same embedding model should be used when indexing document chunks and when embedding user queries. Changing the embedding model without rebuilding the stored vectors can damage retrieval quality.

In this sample project, SentenceTransformers can generate embeddings locally. ChromaDB stores the chunk embeddings together with their text and metadata. When a question arrives, similarity search returns the chunks that are most semantically related to the question.

4. Ollama and Grounded Generation

Ollama runs language models locally and exposes a local API that applications can call. In a RAG application, the retrieved document chunks are formatted as context and sent to the Ollama model together with the user's question.

The prompt should instruct the model to answer from the supplied document context rather than inventing unsupported facts. If the retrieved context does not contain enough information to answer a question, the application should state that the information could not be found in the provided documents.

5. Source-Based Answers

Source references should be derived from the metadata of the retrieved document chunks rather than generated freely by the language model. For example, if a retrieved PDF chunk came from page 2 of a file named architecture.pdf, the application can display architecture.pdf, page 2 as a source. This makes the answer easier to inspect and reduces the risk of fabricated citations.

6. Example Facts for Retrieval

The vector database used in this sample system is ChromaDB. The generation model is served locally through Ollama. The embedding model is sentence-transformers/all-MiniLM-L6-v2. The default retrieval count is four chunks. The application supports PDF and TXT documents. The user interface is built with Streamlit.